



Rendu D1

Team U :

Team members and their roles

Bucur Stefan – PO

Cristea Ana – QA

Iliescu Miruna-Elena – SA

Neata Mihnea-Ioan – Ops

25 october 2025

Plan

1. Périmètre fonctionnel
2. Conception UML
 - 2.1 Le Glossaire
 - 2.2 Le diagramme de cas d'utilisation
 - 2.3 Le(s) diagramme(s) de classes
 - 2.4 Design Patterns
 - 2.5 Le diagramme de séquence
 - 2.6 La maquette
3. Code Quality and Project Management
4. Auto-évaluation

1. Périmètre fonctionnel

Côté	Exigence	Etat	Remarques si besoin	Points forts	Points faibles
Client	C1	DONE	-	Ability to browse dishes from different users	No differentiation between browsing as logged in user or registered user
Client	C2	DONE	-	Restaurant filtration by tags (Cuisine, Dietary tags, price)	-
Client	C5	DONE	-	End-to-end order flow implemented and tested	Could add more edge-case tests
Client	C6	DONE	-	-	Slot is still shown if unavailable, selecting reveals capacity error
Client	C7	DONE	-	Order has to go through different checks before you can pay for it	We could improve the user interface to include even more checks for validation
Restaurant	R2	WIP	So far the restaurant manager doesn't have any interface	-	The manager is only able to mark orders as delivered
Restaurant	R4	NOT IMPLEMENTED	There is no AI implementation	-	-
Restaurant	R5	WIP	-	-	-
Paielement	P1	DONE	Payment step is required before order status changes to PAID.	Payment logic enforces validation; external payments are simulated with redirect URL.	No real integration with external payment providers; only simulated.
Paielement	P2	DONE	After successful payment, the order is saved and linked to the user.	Orders appear in user history only after payment	No rollback if payment fails; limited error handling.
Paielement	P3	DONE	Student credit is implemented as a budget field; payment	Student credit logic is simple and works for multiple orders.	No notification for low credit; no top-up feature.

			deducts from allowance.		
Paielement	P6	DONE	Payment method (student credit or external) is recorded in the order.	Students see payment details; restaurants only see "PAID" status, not method.	Payment method not visible in restaurant view; no audit trail for payment changes
IA/Catalogue	AI2	NOT IMPLEMENTED	There is no AI implementation	-	-

2. Conception UML

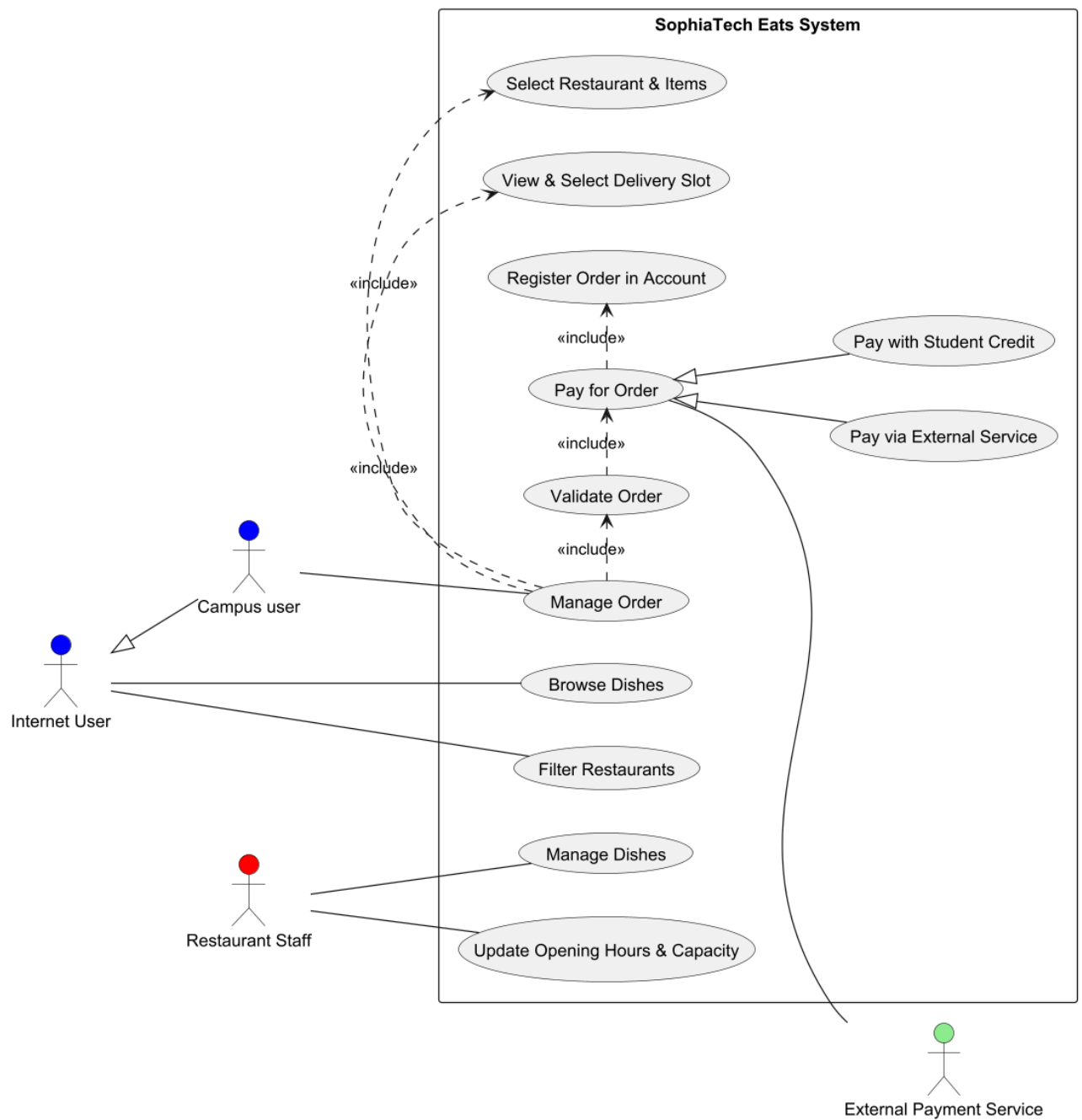
2.1 Le Glossaire

Glossar – SophiaTech Eats

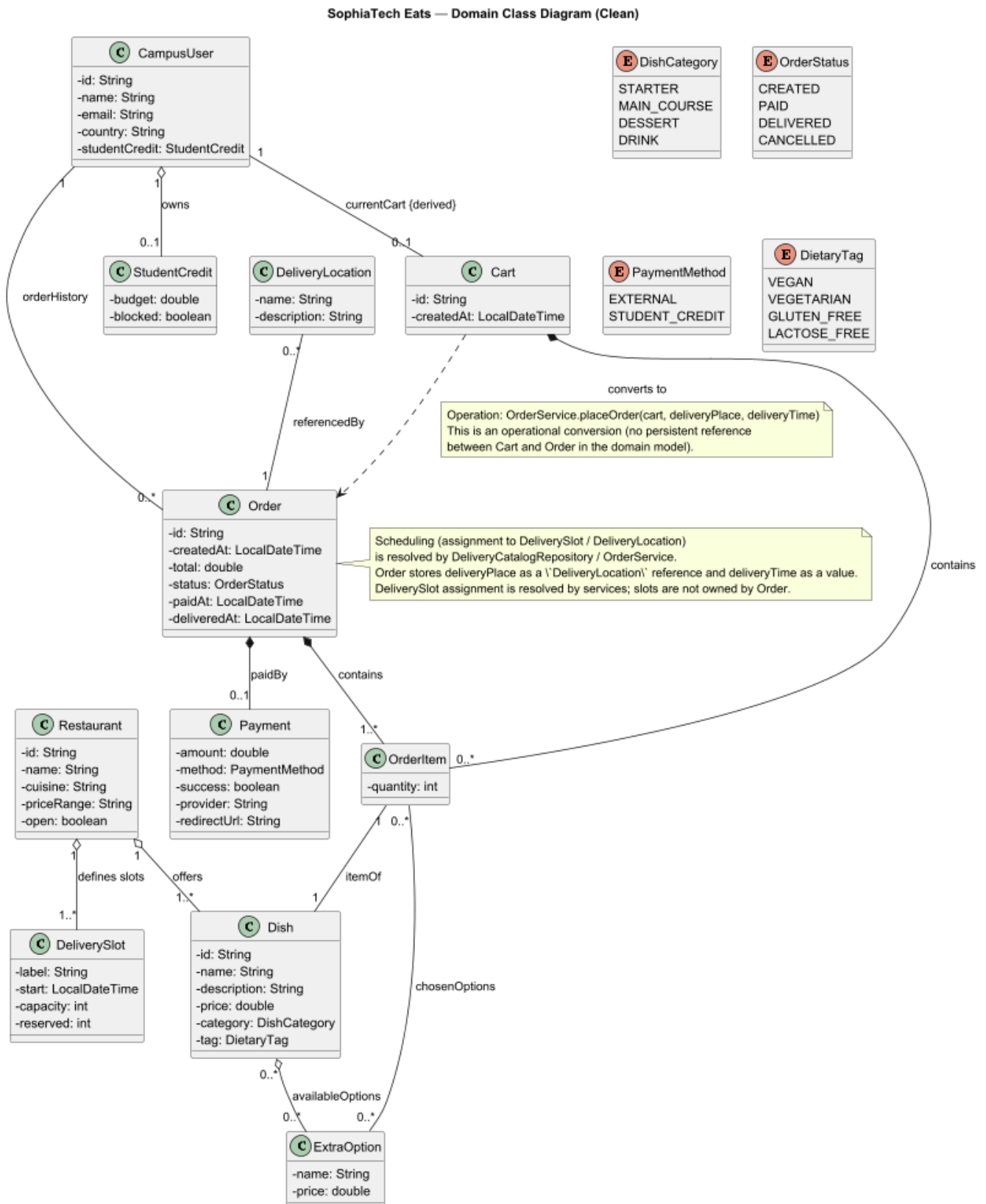
Term	Definition
Order Status* (Changed/Updated)	The state of an order's lifecycle. The currently implemented states are: CREATED (order created but not yet paid), PAID (payment confirmed), DELIVERED (order delivered to the user), and CANCELLED (order cancelled).
Handler* (New)	Architectural component responsible for receiving HTTP requests (e.g., from a UI), extracting data, and calling the appropriate Service to execute logic. Acts as a Controller in the layered architecture.

Service* (New)	Architectural layer containing the business logic (business rules). The Service coordinates operations, validates data, and uses Repositories to access or save data. For example, OrderService validates slot capacity and applies payment strategies.
Repository* (New)	Component that abstracts data access, separating business logic from the storage mechanism (e.g., in-memory or database). Services use Repositories to save, read, or delete domain objects like Order or Restaurant .
Strategy Pattern (Payment)* (New)	A design pattern used in OrderService to handle different payment methods (P1, P3). Each method (EXTERNAL , STUDENT_CREDIT) is a separate "strategy," implemented as a PaymentProcessor , allowing new methods to be added without modifying the service.
PaymentProcessor* (New)	The specific term used in the code (within OrderService) to represent a payment Strategy . It is a functional interface defining a process(Order, CampusUser) method.
Observer Pattern (Cart)* (New)	A design pattern used in CartService . When the cart changes, observers (CartListener) are notified, allowing dynamic updates like recalculating available delivery slots.

2.2 Le diagramme de cas d'utilisation



2.3 Le(s) diagramme(s) de classes

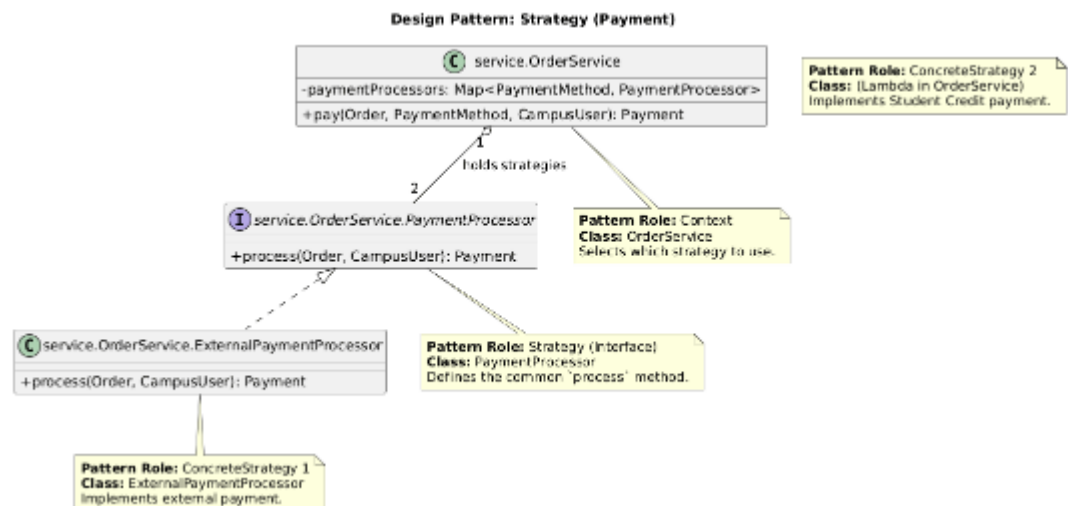


2.4 Design Patterns

Implemented Patterns

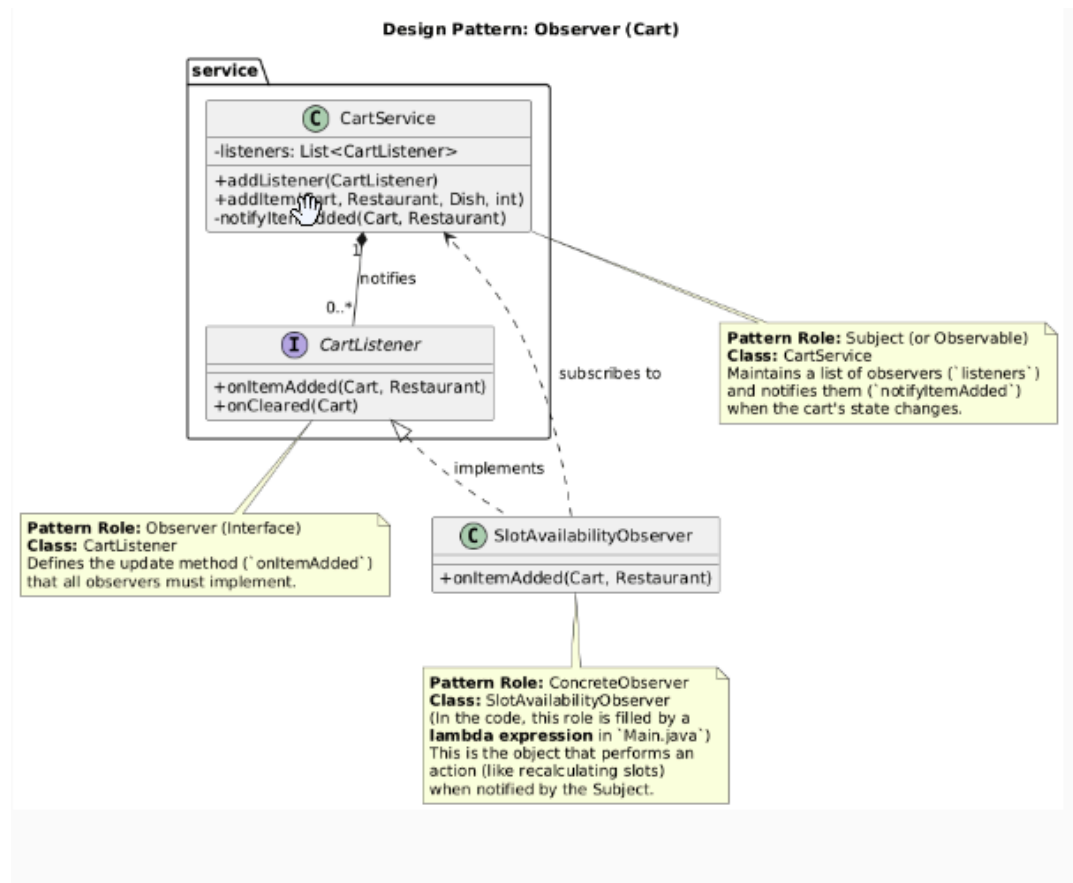
1. Strategy Pattern

- **Context:** Payment handling (requirement P1–P3).
- **Implementation:**
 - The PaymentStrategy interface defines a method processPayment(Order order) implemented by:
 - StudentCreditStrategy
 - ExternalPaymentStrategy
 - OrderService selects which strategy to use based on the user's chosen payment method.
- **Benefits:**
 - Simplifies the addition of new payment types (e.g., vouchers, coupons).
 - Makes unit testing easier since strategies can be mocked independently.



2. Observer Pattern

- **Context:** Cart management and future UI synchronization.
- **Implementation:**
 - The **Cart** class acts as the **Subject**, maintaining a list of observers.
 - Observers are notified when the cart changes.
- **Benefits:**
 - Promotes **loose coupling** between the domain logic and future presentation/UI.



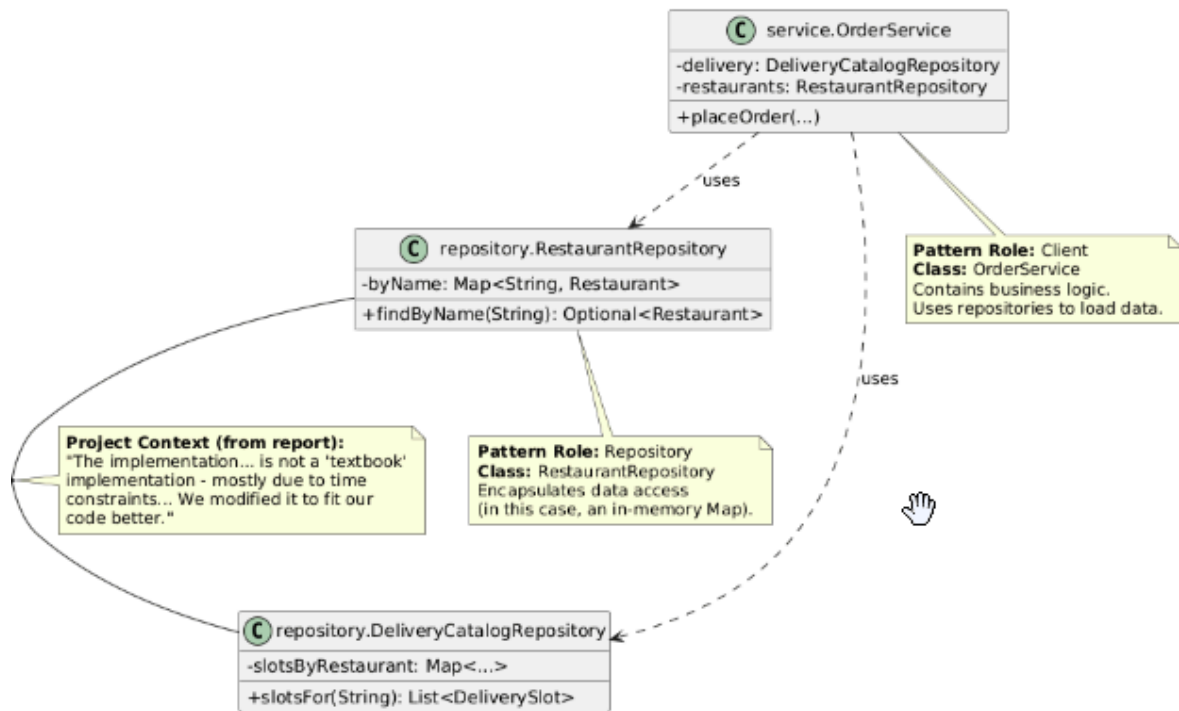
3. Repository Pattern

- **Implementation/Logic:** Interfaces such as [OrderRepository](#), [RestaurantRepository](#), and [UserRepository](#) abstract away data access details.
- The implementation of repository is not a "textbook" implementation - mostly due to time constraints and we modified it to fit our code better.

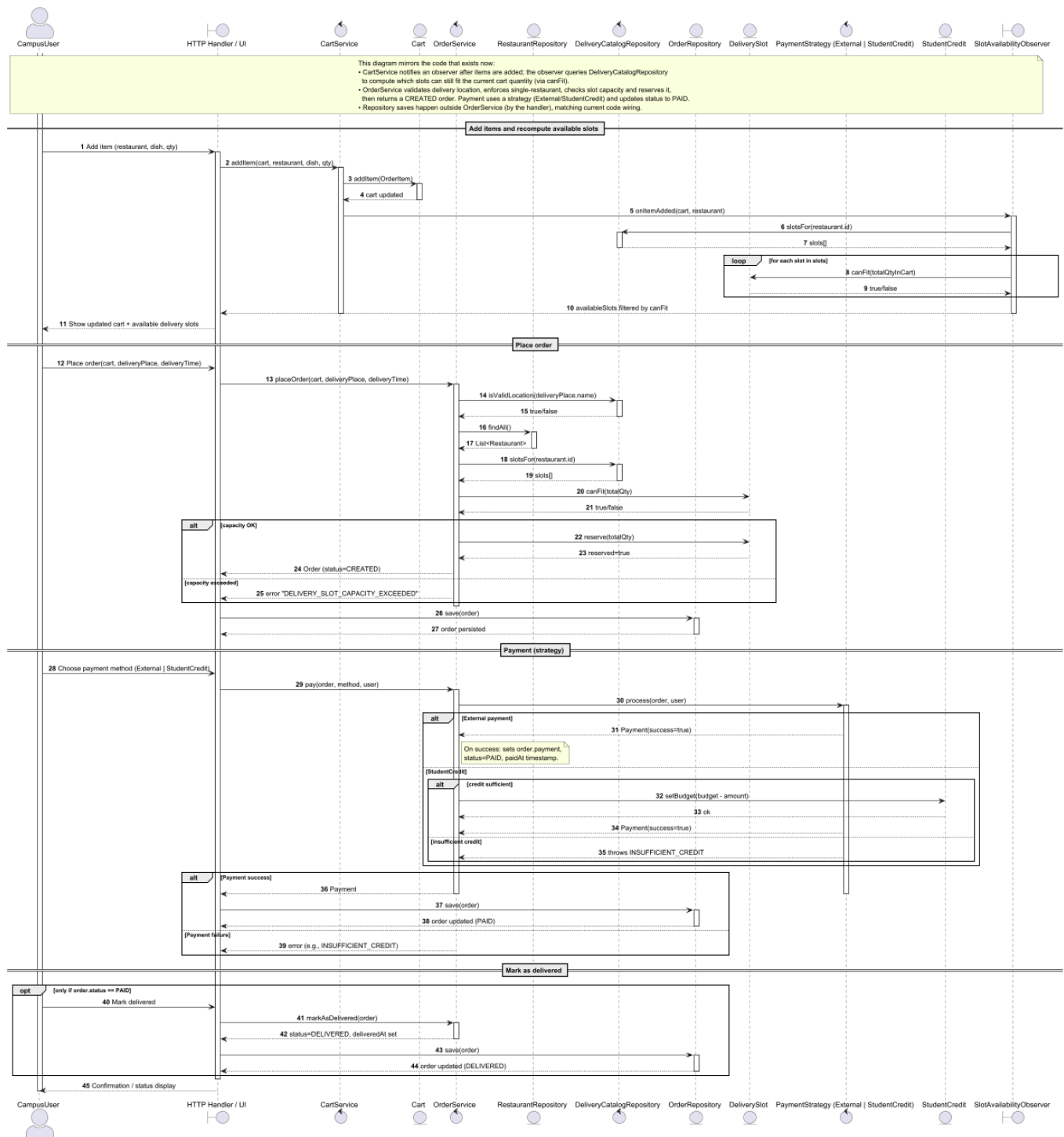
We didn't go more in depth with the design pattern usage due to lack of time. A good idea would have been to create a factory/builder for services and handlers (when they will be implemented), but as it was mentioned, time kept us from it so we went with a more simplistic approach.

The added diagram is fairly simplistic (as is the case for the others as well) - see the diagram further down.

Design Pattern: Repository (Pragmatic)



2.5 Le diagramme de séquence

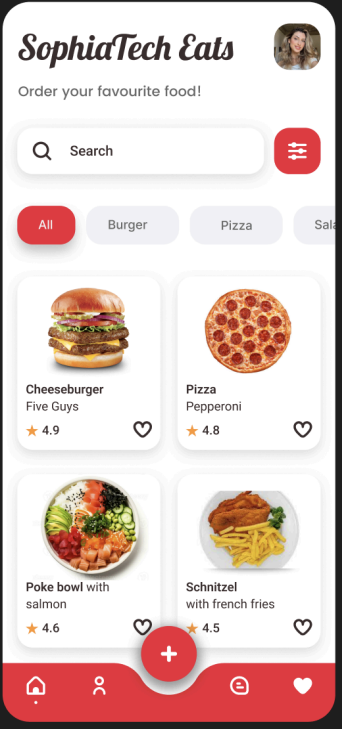
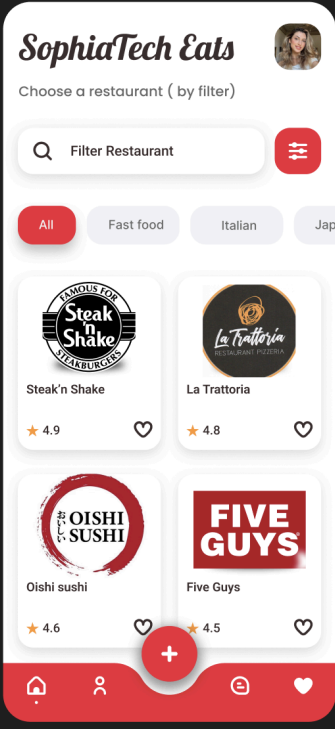


2.6 La maquette

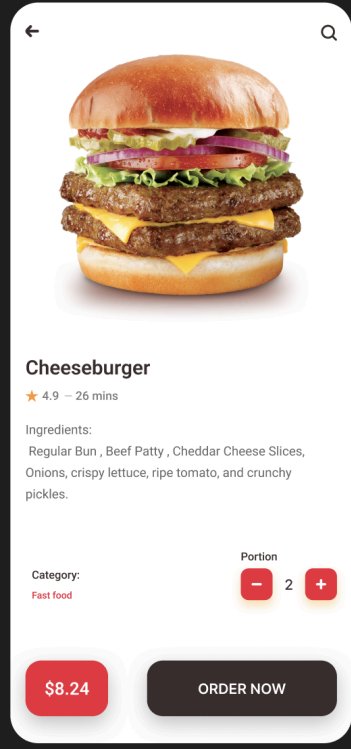
Splash screen



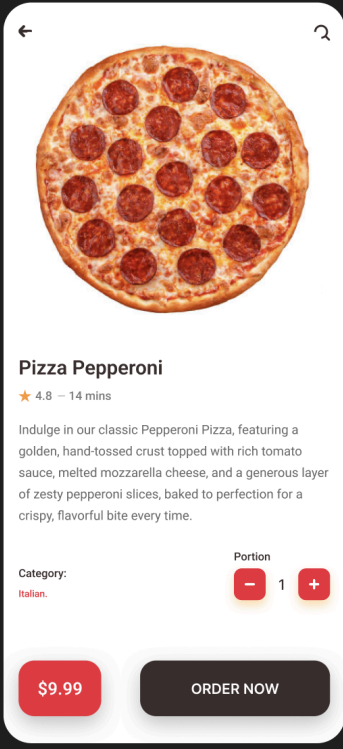
home



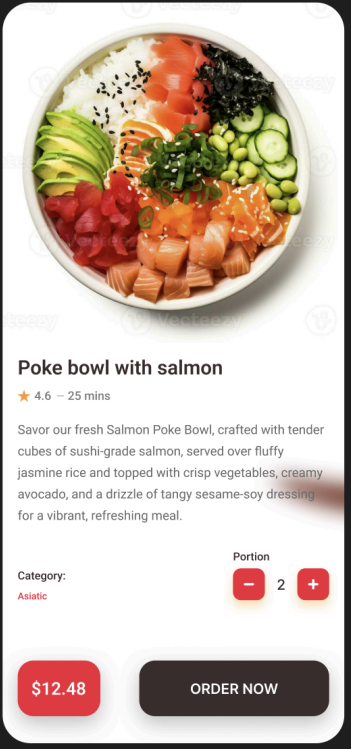
product 1

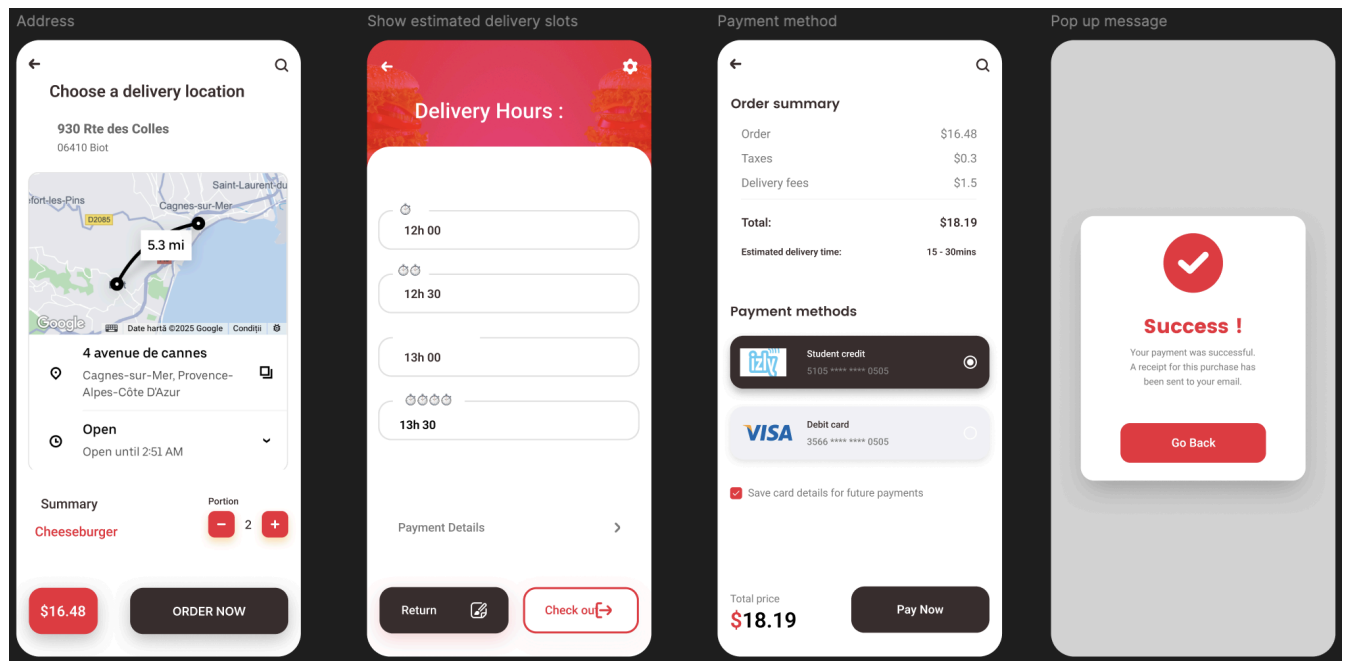


product 2



product 3





4. Code Quality and Project Management

Testing Types and Coverage

- **Unit Tests:**
 - Implemented for core services ([OrderServiceTest](#), [PaymentServiceTest](#)) using JUnit.
 - Covers simple cases such as order validation, payment success/failure, and credit deduction.
 - Aim for ~75% method coverage.
- **Integration+Behavioral Tests (Cucumber+Gherkin)**
 - Validated end-to-end order flows (C1 -> C7) and payment integration.
 - Implemented for acceptance scenarios (e.g., "Place a successful order").
 - Use Gherkin syntax to describe expected system behavior in business terms.
- **Test Quality:**
 - Focused on **determinism and readability**.
 - Mocked dependencies ensure test isolation.

Code Quality and Architecture

- **Layered Structure:** Handler(unused as of now since we only have the backend part)
-> Service -> Repository -> Domain.
This structure enforces separation of concerns and scalability.
- **Clean Code Practices:**
 - Consistent naming, single responsibility per class, and meaningful exception handling.
 - Avoided hard-coded values; used configuration constants.
- **Technical Debt:**
 - Some services (e.g., [RestaurantService](#)) remain partly monolithic and could be decomposed into smaller units.

Project Management and Automation

Branching Strategy:

- [main](#) for stable versions, [specific branch](#) for each member of the team, but we will most likely switch to a feature* type of branching - a branch for each user story.

Automation:

- GitHub Actions runs a build test automatically whenever the main branch is changed.

Documentation:

- Kanban board tracks tasks.
- Each commit message clearly references features or bug fixes.

Collaboration:

- Work evenly distributed among roles: PO (product logic), QA (tests), SA (architecture), Ops (deployment & integration).

5. Self-Evaluation

Team Reflection/ Auto-evaluation

Our team demonstrated **strong collaboration** and clear division of responsibilities throughout the project:

- We adopted **agile-inspired iterations**, focusing first on stable domain modeling, then adding logic and tests progressively.
- Our greatest success was the **end-to-end order flow**, functional and validated by automated tests.
- The main challenge was **time management**, especially balancing test coverage with implementation speed.

Lessons Learned

- Importance of **test-driven development (TDD)** to catch regressions early.
- Necessity of clear **interface definitions** before parallel work begins.
- Communication should be better - especially when working on similar user stories/parts of code, as well as when we want to merge into other branches.

Areas for Improvement

- More robust **error handling** and testing more would be ideal.
- Earlier design of the **restaurant management interface** (R2-R5) could have improved consistency.

Bucur Stefan : 105

Cristea Ana : 90

Iliescu Miruna : 106

Neata Mihnea : 99